

Competitive Programming

Section 12 Editorial: Stacks, Queues, And Deques

Competitive Programming Course

2026-07-21

Table of contents

1	Editorial overview	4
2	A. Next Greater	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	5
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	6
2.8	Full C++ solution	7
2.9	Full Python solution	7
2.10	Common wrong answers	8
2.11	Tests that matter	8
3	B. Sliding Window Maximum	9
3.1	Restatement	9
3.2	Constraints and traps	9
3.3	Observations	9
3.4	Sample walk-through	9
3.5	Algorithm	10
3.6	Correctness proof	10
3.7	Complexity	10
3.8	Full C++ solution	11
3.9	Full Python solution	12
3.10	Common wrong answers	12
3.11	Tests that matter	13
4	C. Shortest Subarray At Least K	14
4.1	Restatement	14
4.2	Constraints and traps	14
4.3	Observations	14
4.4	Sample walk-through	14
4.5	Algorithm	15
4.6	Correctness proof	15
4.7	Complexity	16
4.8	Full C++ solution	16
4.9	Full Python solution	17
4.10	Common wrong answers	18
4.11	Tests that matter	18
5	Problem D: Bounded Spread Subarrays	19
5.1	Restatement	19
5.2	Observation and trace	19

5.3	Algorithm	19
5.4	Correctness proof	20
5.5	Complexity	20
5.6	Implementation traps and tests	20
5.7	Full C++ solution	21
5.8	Full Python solution	22
6	Stress testing	23
7	What to carry forward	24

1 Editorial overview

The section has three problems about maintaining useful candidates.

The main proof idea is dominance:

If candidate A is always at least as good as candidate B in every future situation, candidate B can be deleted permanently.

That is why all three solutions are linear after reading input.

2 A. Next Greater

2.1 Restatement

For every position i , find the first value to the right that is strictly greater than $a[i]$. If no such value exists, output -1 .

2.2 Constraints and traps

The total n is large, so an $O(n^2)$ scan is too slow.

Common traps:

- using greater-or-equal instead of strictly greater;
- scanning left to right without enough information;
- forgetting duplicate values.

2.3 Observations

Scan from right to left. When processing $a[i]$, the stack contains values to the right of i that are still possible next-greater answers.

If the stack top is $\leq a[i]$, it cannot answer i , and it cannot answer any earlier position either, because $a[i]$ is closer and at least as large.

Therefore pop all values $\leq a[i]$.

Then the stack top, if it exists, is the first greater value to the right.

2.4 Sample walk-through

Array:

```
2 1 3 2 5
```

Trace:

```
i=4, x=5: stack=[]      ans=-1, push 5
i=3, x=2: stack=[5]     ans=5,  push 2
i=2, x=3: pop 2, [5]    ans=5,  push 3
i=1, x=1: stack=[5,3]   ans=3,  push 1
i=0, x=2: pop 1, [5,3]  ans=3,  push 2
```

Answer:

```
3 3 5 5 -1
```

2.5 Algorithm

```
answer = [-1] * n
stack = empty

for i from n-1 down to 0:
    while stack not empty and stack.top <= a[i]:
        pop stack
    if stack not empty:
        answer[i] = stack.top
    push a[i]
```

2.6 Correctness proof

Lemma 1. When processing i , every value popped from the stack cannot be the next greater value for position i or any earlier position.

Proof. A popped value y satisfies $y \leq a[i]$. It is not greater than $a[i]$, so it cannot answer position i . For any earlier position $j < i$, $a[i]$ is closer to j than y is, and $a[i] \geq y$. If y were greater than $a[j]$, then $a[i]$ would also be greater than $a[j]$ and would appear earlier to the right, so y could not be the first greater value. $\square$

Lemma 2. After popping, if the stack is nonempty, its top is the first value to the right that is strictly greater than $a[i]$.

Proof. All values on top that are not strictly greater have been removed. The stack keeps candidates in the order they would be encountered from left to right among the surviving right-side elements; the top is the closest surviving greater value. By Lemma 1, removed values cannot be valid answers. $\square$

Theorem. The algorithm outputs the correct next greater value for every position.

Proof. By Lemma 2, each position receives the correct stack top if a greater value exists. If the stack is empty, all right-side values have been either removed as not useful or do not exist, so no greater value exists. $\square$

2.7 Complexity

Each element is pushed once and popped at most once.

```
Time:   $O(n)$ 
Memory:  $O(n)$ 
```

2.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> solve_case(const vector<long long> &a) {
    vector<long long> ans(a.size(), -1);
    vector<long long> st;
    for (int i = (int)a.size() - 1; i >= 0; --i) {
        while (!st.empty() && st.back() <= a[i]) {
            st.pop_back();
        }
        if (!st.empty()) {
            ans[i] = st.back();
        }
        st.push_back(a[i]);
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        vector<long long> ans = solve_case(a);
        for (int i = 0; i < n; ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}
```

2.9 Full Python solution

```
import sys

def solve_case(a: list[int]) -> list[int]:
```

```

ans = [-1] * len(a)
stack: list[int] = []
for i in range(len(a) - 1, -1, -1):
    while stack and stack[-1] <= a[i]:
        stack.pop()
    if stack:
        ans[i] = stack[-1]
    stack.append(a[i])
return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        a = data[idx:idx + n]
        idx += n
        out.append(" ".join(map(str, solve_case(a))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

2.10 Common wrong answers

- Not popping equal values.
- Returning the maximum greater value rather than the first greater value.
- Using a nested loop.

2.11 Tests that matter

```

all equal:      5 5 5 -> -1 -1 -1
increasing:    1 2 3 -> 2 3 -1
decreasing:    3 2 1 -> -1 -1 -1

```

3 B. Sliding Window Maximum

3.1 Restatement

For every contiguous subarray of length k , output its maximum value.

3.2 Constraints and traps

The total n is large. Recomputing each maximum from scratch costs $O(nk)$, which is too slow.

Common traps:

- storing values instead of indices;
- forgetting to remove expired indices;
- using `list.pop(0)` in Python.

3.3 Observations

Keep a deque of indices.

The deque stores only useful candidates:

- indices increase from front to back;
- values decrease from front to back;
- the front is always the maximum of the current window.

When new index i arrives, every index at the back with value $\leq a[i]$ is dominated by i : it is older and no larger.

3.4 Sample walk-through

Array:

```
1 3 -1 -3 5 3 6 7
k = 3
```

The first full window is $[1, 3, -1]$, and the deque front points to value 3.

When 5 arrives, it removes -3, -1, and 3 from the back because all are smaller and older. Then 5 becomes the only useful maximum candidate.

Output:

```
3 3 5 5 6 7
```

3.5 Algorithm

```
dq = empty deque of indices
answer = []

for i in 0..n-1:
    while dq not empty and dq.front <= i-k:
        pop front
    while dq not empty and a[dq.back] <= a[i]:
        pop back
    push i
    if i+1 >= k:
        answer.append(a[dq.front])
```

3.6 Correctness proof

Lemma 1. The deque contains only indices inside the current window after the expiration step.

Proof. An index is outside the current window exactly when it is at most $i-k$. The algorithm removes such indices from the front. Since indices in the deque are increasing, no later index can expire before the front. $\square$

Lemma 2. Values in the deque are strictly decreasing after the insertion step.

Proof. Before inserting i , the deque already has decreasing values. The algorithm removes every back index whose value is $\leq a[i]$, then appends i . Therefore the previous back, if any, has value greater than $a[i]$. $\square$

Lemma 3. The front of the deque is the index of the maximum value in the current window.

Proof. By Lemma 1, every deque index is inside the window. By Lemma 2, the front has value at least every other deque candidate. Any removed index was either expired or dominated by a newer index with value at least as large, so no removed index can be the maximum of the current or future window. $\square$

Theorem. The algorithm outputs every window maximum correctly.

Proof. Whenever a full window exists, Lemma 3 says $a[dq.front]$ is exactly the window maximum. The algorithm appends that value for every window. $\square$

3.7 Complexity

Every index is pushed once and popped at most once from each end.

```
Time:  $O(n)$ 
Memory:  $O(k)$ 
```

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

vector<long long> solve_case(const vector<long long> &a, int k) {
    deque<int> dq;
    vector<long long> ans;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!dq.empty() && dq.front() <= i - k) {
            dq.pop_front();
        }
        while (!dq.empty() && a[dq.back()] <= a[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        if (i + 1 >= k) {
            ans.push_back(a[dq.front()]);
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        vector<long long> ans = solve_case(a, k);
        for (int i = 0; i < (int)ans.size(); ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}
```

3.9 Full Python solution

```
from collections import deque
import sys

def solve_case(a: list[int], k: int) -> list[int]:
    dq: deque[int] = deque()
    ans: list[int] = []
    for i, x in enumerate(a):
        while dq and dq[0] <= i - k:
            dq.popleft()
        while dq and a[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if i + 1 >= k:
            ans.append(a[dq[0]])
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        k = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        out.append(" ".join(map(str, solve_case(a, k))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

3.10 Common wrong answers

- Not expiring old indices before reading the maximum.
- Using < instead of <= and keeping unnecessary duplicate candidates. This can still be correct, but it changes the invariant.
- Outputting before the first full window exists.

3.11 Tests that matter

```
k = 1      -> output the array  
k = n      -> one maximum  
duplicates -> stable handling of equal values
```

4 C. Shortest Subarray At Least K

4.1 Restatement

The array may contain negative numbers. Find the shortest nonempty contiguous subarray with sum at least K, or print -1 if none exists.

4.2 Constraints and traps

Because values can be negative, ordinary two pointers do not work. Extending the right endpoint can decrease the sum, and moving the left endpoint can increase it.

Use prefix sums:

```
sum(l, r-1) = pref[r] - pref[l]
```

4.3 Observations

For each right prefix index i , we need an earlier index j with:

```
pref[i] - pref[j] >= K
```

For a fixed i , smaller $\text{pref}[j]$ is better. If two candidate indices $j_1 < j_2$ satisfy:

```
pref[j1] >= pref[j2]
```

then j_1 is dominated. It is earlier and has a larger or equal prefix, so it produces no larger sum and a longer length than j_2 .

Keep candidate prefix indices in increasing prefix-sum order.

4.4 Sample walk-through

Array:

```
2 -1 2 -1 2 4
K = 5
```

Prefix sums:

```
0 2 1 3 2 4 8
```

At prefix index 6, value 8, a candidate prefix value 2 at index 4 gives:

```
8 - 2 = 6
length = 6 - 4 = 2
```

This is the subarray:

```
2 4
```

Answer:

```
2
```

4.5 Algorithm

```
pref[0] = 0
for each value:
    append next prefix sum

answer = n + 1
dq = empty deque of prefix indices

for i from 0 to n:
    while dq not empty and pref[i] - pref[dq.front] >= K:
        answer = min(answer, i - dq.front)
        pop front

    while dq not empty and pref[dq.back] >= pref[i]:
        pop back

    push i

if answer == n + 1:
    print -1
else:
    print answer
```

4.6 Correctness proof

Lemma 1. If $j_1 < j_2$ and $\text{pref}[j_1] \geq \text{pref}[j_2]$, then j_1 is never a better left endpoint than j_2 for any future right endpoint.

Proof. For any future $i > j_2$, the sum using j_2 is:

```
pref[i] - pref[j2]
```

which is at least:

```
pref[i] - pref[j1]
```

because $\text{pref}[j2] \leq \text{pref}[j1]$. The length $i-j2$ is also shorter than $i-j1$. So $j2$ is at least as good in sum and better in length. $\square$

Lemma 2. The deque stores prefix indices with strictly increasing prefix values.

Proof. Before pushing i , the algorithm removes every back index whose prefix value is at least $\text{pref}[i]$. Therefore the remaining back, if any, has a smaller prefix value than $\text{pref}[i]$. $\square$

Lemma 3. When the front of the deque forms a valid subarray with current index i , popping it after updating the answer is safe.

Proof. Once $\text{pref}[i] - \text{pref}[\text{dq.front}] \geq K$, the subarray ending at i is valid. For any later right endpoint, using the same front index gives a longer subarray. Therefore the best answer involving that front index has already been recorded. $\square$

Theorem. The algorithm returns the length of the shortest subarray with sum at least K .

Proof. Lemma 1 justifies removing dominated back candidates. Lemma 3 justifies removing valid front candidates after recording their best possible length. Every undominated candidate remains in the deque until it is either used or dominated. Therefore every possible optimal left endpoint is considered at the first right endpoint where it can give a valid subarray, and the minimum recorded length is optimal. $\square$

4.7 Complexity

Each prefix index is pushed once and popped at most once from each side.

```
Time:  $O(n)$   
Memory:  $O(n)$ 
```

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int solve_case(const vector<long long> &a, long long target) {
    vector<long long> pref(a.size() + 1, 0);
    for (int i = 0; i < (int)a.size(); ++i) {
        pref[i + 1] = pref[i] + a[i];
    }

    int ans = (int)a.size() + 1;
    deque<int> dq;
```

```

for (int i = 0; i < (int)pref.size(); ++i) {
    while (!dq.empty() && pref[i] - pref[dq.front()] >= target) {
        ans = min(ans, i - dq.front());
        dq.pop_front();
    }
    while (!dq.empty() && pref[dq.back()] >= pref[i]) {
        dq.pop_back();
    }
    dq.push_back(i);
}
return ans == (int)a.size() + 1 ? -1 : ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        long long target;
        cin >> n >> target;
        vector<long long> a(n);
        for (long long &x : a) cin >> x;
        cout << solve_case(a, target) << '\n';
    }

    return 0;
}

```

4.9 Full Python solution

```

from collections import deque
import sys

def solve_case(a: list[int], target: int) -> int:
    pref = [0]
    for x in a:
        pref.append(pref[-1] + x)

    ans = len(a) + 1
    dq: deque[int] = deque()
    for i, value in enumerate(pref):
        while dq and value - pref[dq[0]] >= target:
            ans = min(ans, i - dq.popleft())
        while dq and pref[dq[-1]] >= value:

```

```

        dq.pop()
        dq.append(i)
    return -1 if ans == len(a) + 1 else ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        target = data[idx + 1]
        idx += 2
        a = data[idx:idx + n]
        idx += n
        out.append(str(solve_case(a, target)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4.10 Common wrong answers

- Using a positive-only sliding window on arrays with negative numbers.
- Forgetting the prefix sum at index 0.
- Popping the back with the wrong inequality.
- Using int for prefix sums in C++.

4.11 Tests that matter

```

negative values only
K <= 0 with nonempty subarray requirement
answer is a single element
answer does not exist

```

5 Problem D: Bounded Spread Subarrays

5.1 Restatement

Count the nonempty contiguous subarrays whose maximum value minus minimum value is at most k . Array values may be negative; only the spread is constrained.

This is the section's blind-practice exercise. It needs two monotonic deques at once: one to expose the current maximum and one to expose the current minimum.

5.2 Observation and trace

For a fixed right endpoint, if $[\text{left}, \text{right}]$ has acceptable spread, every shorter suffix ending at right is also acceptable: removing values cannot increase the maximum or decrease the minimum. Therefore after shrinking to a valid window, all $\text{right} - \text{left} + 1$ starts from left through right contribute.

Keep decreasing values in the maximum deque and increasing values in the minimum deque. Store indices, so expired positions can be removed when left moves.

For 1 3 2 5 and $k=2$:

right	value	valid window after shrinking	new subarrays	answer
1		[1]	1	1
3		[1,3]	2	3
2		[1,3,2]	3	6
5		[5]	1	7

At the last step the spread exceeds two until the left pointer has passed the earlier values.

5.3 Algorithm

```
left = 0
max_deque = empty decreasing deque of indices
min_deque = empty increasing deque of indices
answer = 0

for right from 0 to n-1:
    remove dominated back indices from max_deque
    append right to max_deque
    remove dominated back indices from min_deque
    append right to min_deque
```

```

while a[max_deque.front] - a[min_deque.front] > k:
    remove left from either deque when it is at the front
    left += 1

answer += right-left+1

```

5.4 Correctness proof

Lemma 1. After each update, the front of `max_deque` is an index of the maximum value in the current window, and the front of `min_deque` is an index of its minimum value.

Proof. When a new value arrives, every older maximum-deque back value no larger than it is dominated: the new index lasts longer and is at least as large. Removing those backs preserves all possible future maxima. The minimum argument is symmetric for values no smaller than the new value. Expired front indices are removed when the left endpoint passes them. Thus the surviving fronts are exactly the needed extrema. $\square$

Lemma 2. After the shrinking loop, `[left,right]` has spread at most `k`, and every subarray ending at `right` that starts before `left` is invalid.

Proof. Lemma 1 makes the loop condition exactly the current window's spread. The loop stops only when it is at most `k`. Each discarded left index belongs to a window whose spread was greater than `k`; extending that discarded window farther left cannot reduce its spread, so any earlier start is invalid. $\square$

Theorem. The algorithm returns the required number of subarrays.

Proof. By Lemma 2, exactly the `right-left+1` suffixes ending at each right endpoint are valid. The algorithm adds this number once per endpoint. Every nonempty subarray has one right endpoint, so all and only valid subarrays are counted once. $\square$

5.5 Complexity

Each index enters and leaves each deque at most once. Time is $O(n)$ per test case and memory is $O(n)$ in the worst case. The answer can be $n(n+1)/2$, so use `long long` in C++.

5.6 Implementation traps and tests

- Store indices, not just values, so you can remove expired endpoints.
- Use `<=` for the decreasing maximum deque and `>=` for the increasing minimum deque; retaining only the newest equal value is safe.
- Shrink before adding the count for the current right endpoint.
- Negative values are valid, so do not use a sum-based sliding window.

Test `k=0`, all equal values, alternating extreme values, a one-element array, and the all-equal 200000-element case that forces a 64-bit answer. The random generator uses direct quadratic enumeration on small arrays.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t;
    cin >> t;
    while (t--) {
        int n;
        long long k;
        cin >> n >> k;

        vector<long long> values(n);
        for (long long &value : values) cin >> value;

        deque<int> max_deque, min_deque;
        int left = 0;
        long long answer = 0;
        for (int right = 0; right < n; ++right) {
            while (!max_deque.empty() &&
                values[max_deque.back()] <= values[right]) {
                max_deque.pop_back();
            }
            max_deque.push_back(right);
            while (!min_deque.empty() &&
                values[min_deque.back()] >= values[right]) {
                min_deque.pop_back();
            }
            min_deque.push_back(right);
            while (values[max_deque.front()] - values[min_deque.front()] > k) {
                if (max_deque.front() == left) {
                    max_deque.pop_front();
                }
                if (min_deque.front() == left) {
                    min_deque.pop_front();
                }
                ++left;
            }
            answer += right - left + 1;
        }
        cout << answer << '\n';
    }
    return 0;
}
```

5.8 Full Python solution

```
from collections import deque
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    index = 1
    out: list[str] = []
    for _ in range(data[0]):
        n = data[index]
        k = data[index + 1]
        index += 2
        values = data[index:index + n]
        index += n

        max_deque: deque[int] = deque()
        min_deque: deque[int] = deque()
        left = 0
        answer = 0

        for right, value in enumerate(values):
            while max_deque and values[max_deque[-1]] <= value:
                max_deque.pop()
            max_deque.append(right)

            while min_deque and values[min_deque[-1]] >= value:
                min_deque.pop()
            min_deque.append(right)

            while values[max_deque[0]] - values[min_deque[0]] > k:
                if max_deque[0] == left:
                    max_deque.popleft()
                if min_deque[0] == left:
                    min_deque.popleft()
                left += 1

            answer += right - left + 1

        out.append(str(answer))

    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()
```

6 Stress testing

For next greater:

```
For each i, scan j=i+1..n-1 and take the first greater value.
```

For sliding maximum:

```
For each window, compute max directly.
```

For shortest subarray:

```
Try every left endpoint and extend right while summing.
```

The local generators use these brute-force checks on small arrays.

7 What to carry forward

The important habit is to identify dominance.

Ask:

- What makes an older candidate useless?
- Should equality remove the older candidate?
- Does the structure need values, indices, or both?
- When does a candidate expire?
- Can each candidate be pushed and popped only once?

These questions are the bridge from simple stacks and queues to later graph BFS, DP optimization, and advanced monotonic data structures.